INDIAN INSTITUTE OF TECHNOLOGY, KHARAGPUR



Quantum Circuit Simulator

by

Kshitiz Anand

A thesis submitted in partial fulfillment for the
degree of Bachelors of Technology

Under the guidance of
Prof. Sarang Pendharker
Department Of Electronics and Electrical Communication Engineering

April 2023

Declaration

I, Kshitiz Anand, declare that this thesis titled, 'Quantum Circuit Simulator' and the work displayed in it are mine. I confirm the following:

- The entirety of this work was completed while pursuing a Bachelor's degree at this university.
- The work has not been submitted to any other Institute for any degree or diploma.
- I have complied with the standards and recommendations outlined in the Institute's Ethical Code of Conduct.
- All significant aid sources have been acknowledged.

Date: April 28, 2023 *(Kshitiz Anand)*

Place: Kharagpur *(19EC10038)*

INDIAN INSTITUTE OF TECHNOLOGY, KHARAGPUR

Department Of Electronics and Electrical Communication Engineering

Certificate



This is to certify that the project report entitled "Quantum Circuit Simulator" submitted by **Kshitiz Anand** (*Roll No. 19EC10038*) to Indian Institute of Technology Kharagpur towards partial fulfilment of requirements for the award of degree of Bachelors of Technology in Electronics and Electrical Communication Engineering is a record of bonafide work carriend out by him under my supervision and guidance during Spring Semester, 2022-23.

Date: 28/04/2023

Place: Kharagpur

Prof. Sarang Pendharker
Department of Electronics and Electrical Communication Engineering
(Indian Institute of Technology, Kharagpur, India)

INDIAN INSTITUTE OF TECHNOLOGY, KHARAGPUR

Abstract

Prof. Sarang Pendharker
Department Of Electronics and Electrical Communication Engineering

Bachelors of Technology

by Kshitiz Anand

This project aims at developing a Python library that has the effective tools to simulate the behaviour of a quantum circuit, the qubits, and the gate operations on a qubit under ideal and non-ideal circumstances. The quantum computation field is an important work in progress. With the development of quantum computers, we can do tasks that would have taken the current classical computer years, in a short amount of time. For the testing of the quantum computer, it is important to have a proper testing unit with which we can match the computation results. This library can also be used a simple toolkit to understand the fundamentals of a quantum circuit and how can we use quantum computing to our advantage.

Acknowledgements

I would like to express my sincere gratitude to my supervisor Prof. Sarang Pendharker, for his extraordinary leadership and support, without which this research would not have been a success.

I would also like to thank the Department of Electronics and Electrical Communication Engineering, IIT Kharagpur for the numerous resources and assistance provided to complete this research.

Contents

Declaration i

Certificate ii

Abstract iii

Acknowledgements iv

1 Introduction 1
- 1.1 Church-Turing Theory . . . 1
- 1.2 Qubits . . . 2
  - 1.2.1 Quantum Entanglement . . . 3
  - 1.2.2 Gate Operations On Qubits . . . 3
- 1.3 Losses In Real Life Quantum Circuits . . . 4
  - 1.3.1 Driveline Dissipation . . . 4
  - 1.3.2 Driveline Noise . . . 4
  - 1.3.3 Instrinsic . . . 4
  - 1.3.4 Readout . . . 4

2 Circuit Implementation 5
- 2.1 Single Qubit System . . . 5
- 2.2 Double Qubit System . . . 6
- 2.3 Multiple Qubit Systems . . . 7
  - 2.3.1 Setting The States Of Qubits . . . 8
  - 2.3.2 Basic Gate Operations . . . 8
- 2.4 Non Ideal Qubits . . . 10
  - 2.4.1 Effect Of Decoherence On The Qubit . . . 11
  - 2.4.2 X-Gate Operation With Decoherence . . . 12
  - 2.4.3 U Gate Operation With Decoherence . . . 13

3 Simulations and Results 14
- 3.1 Single Qubit System . . . 14
- 3.2 Double Qubit System . . . 14
- 3.3 Multiple Qubit System . . . 15
- 3.4 Non Ideal Qubit . . . 16
  - 3.4.1 Decoherence Decay . . . 16

3.4.2 U Gate Operation 17
3.4.3 X-Gate Operation 18
3.4.4 H-Gate Operation 19

Chapter 1

Introduction

"The disruptive potential of quantum technology will make the change of the Internet era look like a small bump in the road!"- Kevin Coleman

The first idea of a quantum computer ranges back to 1970 when the renowned physicist Richard Feynman found out that to simulate quantum physics calculations on a classical computer, it would require a lot of time, almost exponential time for a given number of atoms. In a paper published in 1982, he postulated that to simulate quantum systems, we would need to build quantum computers.

The real idea of quantum computing comes from the inherited probabilistic nature of all quantum events. So, we cannot state that the qubit is surely in state 1 or state 0, there is always a probabilistic aspect to it and the state of the qubit is referred to as a superposition of state 1 and state 0.

Quantum computing is a type of computation whose operations can harness the phenomena of quantum mechanics, such as superposition, interference, and entanglement. Devices that perform quantum computations are known as quantum computers. Though current quantum computers are too small to outperform usual (classical) computers for practical applications, larger realizations are believed to be capable of solving certain computational problems, such as integer factorization (which underlies RSA encryption), substantially faster than classical computers. The study of quantum computing is a subfield of quantum information science.

1.1 Church-Turing Theory

A Turing machine is a mathematical model of computation describing an abstract machine that manipulates symbols on a strip of tape according to a table of rules. Despite the model's simplicity, it can implement any computer algorithm.

The machine operates on an infinite memory tape divided into discrete cells, each of which can hold a single symbol drawn from a finite set of symbols called the alphabet of the machine. It has a head that, at any point in the machine's operation, is positioned over one of these cells, and a state selected from a finite set of states. At each step of its operation, the head reads the symbol in its cell. Then, based on the symbol and the machine's own present state, the machine writes a symbol into the same cell, and moves the head one step to the left or the right, or halts the computation. The choice of which replacement symbol to write and which direction to move is based on a finite table that specifies what to do for each combination of the current state and the symbol that is read.

The Church-Turing theory states that any computation problem can be solved on any computer that we hope to build in the future if and only if it can be solved on the turing machine.

A stronger quantum version of the Church-Turing theory states that ''a quantum turing machine can efficiently simulate any realistic model of computation".

1.2 Qubits

In quantum computing, a qubit or quantum bit is a basic unit of quantum information, the quantum version of the classic binary bit physically realized with a two-state device. A qubit is a two-state (or two-level) quantum-mechanical system. Examples include the spin of the electron in which the two levels can be taken as spin up and spin down; or the polarization of a single photon in which the two states can be taken to be the vertical polarization and the horizontal polarization. In a classical system, a bit would have to be in one state or the other. However, quantum mechanics allows the qubit to be in a coherent superposition of both states simultaneously, a property that is fundamental to quantum mechanics and quantum computing. The state of the qubit evolves through the various gates operations that is done on it.

For a system of n components, a complete description of its state in classical physics requires only n bits, whereas in quantum physics it requires 2^n complex numbers.

In quantum mechanics, the general state of qubit can be represented by a linear superposition of its two orthonormal basis state vectors. The basis state vectors can be represented as follows:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

A qubit state represented as the linear combination of the basis states with values α and β. Here α and β are complex numbers known as the probability amplitudes. When we measure the qubit in the standard basis, according to Born rule, the probability of the outcome ket 0 with value "0" is α^2 and that of the outcome ket 1 with value "1" is β^2. The sum of these probabilities comes out to be 1 as specified by the second axiom of the probability theorem.

1.2.1 Quantum Entanglement

Quantum entanglement is the phenomenon that occurs when a group of particles are generated, interact, or share spatial proximity in a way such that the quantum state of each particle of the group cannot be described independently of the state of the others, including when the particles are separated by a large distance. The topic of quantum entanglement is at the heart of the disparity between classical and quantum physics: entanglement is a primary feature of quantum mechanics not present in classical mechanics.

An important distinguishing feature between qubits and classical bits is that multiple qubits can exhibit quantum entanglement. Quantum entanglement of two or more qubits allows a set of qubits to express higher correlation than is possible in classical systems. Once two qubits are entangled, it is not possible to separate the two qubits' states and operate on them separately. Once two qubits are entangled, they will be stay entangled no matter how far are separated or how much time they are separated, they will still stay entangled and their states will be influenced by each other.

Even though a entangled system of n qubit can hold in 2^n information, when measuring it will finally collapse into one of the n states losing all the extra information that it withheld.

1.2.2 Gate Operations On Qubits

The states of qubits can be altered using quantum gates, analogous to how classical gates alter the state of a classical bit. The application of each quantum gate to a qubit can be represented as a matrix multiplication between the quantum gate operator and the qubit state vector. A quantum computation can be described as a network of quantum logic gates and measurements. However, any measurement can be deferred to the end of quantum computation, though this deferment may come at a computational cost, so most quantum circuits depict a network consisting only of quantum logic gates and no measurements.

The basic one qubit gates are Pauli gates, Hadamard gate, Phased gate, Virtual Z gate, Identity gate, S gate, $S^\dagger$ gate, T gate, $T^\dagger$ gate, U gate. Another basic operation on the qubit can be measurement.

1.3 Losses In Real Life Quantum Circuits

In real life, there are various factors such as decays and losses constantly affecting the state of the quabit. Energy keeps on leaking from the qubit bring its state to the lower energy ket 0 state. These losses depend on a variety of factors such as the temperature in which the qubit is kept, the frequency of the qubit, and the pulses that are used for gate operations. Each loss has its own specific quality factor and hence its own time constant. This helps us in modelling the losses as an exponential function. The equations below give us an idea of how the losses are modelled.

$$\gamma = \frac{1}{T} = \frac{Quality\ Factor}{\omega} \tag{1.1}$$

The evolution of the probability amplitudes can be modelled using the following equations:

$$|\beta'|^2 = |\beta|^2 e^{-\gamma t} \tag{1.2}$$

$$|\alpha'|^2 = 1 + (|\alpha|^2 - 1)e^{-\gamma t} \tag{1.3}$$

The various types of losses are as follows:

1.3.1 Driveline Dissipation

1.3.2 Driveline Noise

1.3.3 Instrinsic

1.3.4 Readout

Chapter 2

Circuit Implementation

The complete library was built from scratch in python. Objects for qubit systems were created with the basic gate operations as methods of the object.

2.1 Single Qubit System

We can create single qubit systems by importing the single_qubit.py file from the library and calling the constructor function passing in the initial state of the qubit as a parameter to the constructor. The value between 0 and 1 is taken as the probability amplitude of the ket 1 state and the other probability amplitude is calculated using the law of total probabilities.

```
def __init__(self, x):
    if(x<=1 and x>=0):
        y = (1 - x**2)**0.5
            self.qubit_state = [[y],[x]]
    else:
        raise Exception("Enter a valid qubit state")
```

One of the most basic gates in quantum circuits is the Pauli X gate, the final output of the x gate operation can be found by multiplying the state vector with the specific matrix for the x gate operation which is given as:

$$[X] = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = |0><1| + |1><0| \tag{2.1}$$

The implementation for the gate in the code is as follows:

```
def xGate(self):
    self.qubit_state = pauliGates.PauliGates.pauli_x_gate(self.qubit_state)


def pauli_x_gate(qubit_state):
    pauli_x_operator = [[0,1],[1,0]]
    output = np.dot(pauli_x_operator, qubit_state)
    return output
```

Similarly all the other basic gates for a single qubit system have been implemented. The list of all the single qubit gates is as follows:

SingleQubit(), printState(), xGate(), yGate(), zGate(), hGate(), measure0(), measure1(), pGate(), virtualzGate(), iGate(), sGate(), sDaggerGate(), tGate(), tDaggerGate(), pGate(), getState()

2.2 Double Qubit System

We can create double qubit systems by importing the double_qubit_system.py file from the library and calling the constructor function for the object. The constructor function takes in two single qubit objects and creates a double qubit system with the specific probability amplitudes. The constructor for the system is as follows:

```
def __init__(self, q1, q2):
    a = q1.qubit_state[0][0]*q2.qubit_state[0][0]
    b = q1.qubit_state[0][0]*q2.qubit_state[1][0]
    c = q1.qubit_state[1][0]*q2.qubit_state[0][0]
    d = q1.qubit_state[1][0]*q2.qubit_state[1][0]
    self.qubit_state = [[a],[b],[c],[d]]
```

One of the most basic double qubit gates in any quantum circuit is the CNOT gates, also known as the controlled not gate. In this gate the first qubit acts as the control qubit and the second qubit is the one on which the NOT operation is applied. The matrix operator for the CNOT gates is given as follows:

$$[CNOT] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{2.2}$$

The implementation for the gate in code is as follows:

```
def cnotGate(self):
    self.qubit_state = cnotGate.cnotGate.cnot_gate(self.qubit_state)

def cnot_gate(qubit_state):
    cnot_operator = [[1,0,0,0],[0,1,0,0],[0,0,0,1],[0,0,1,0]]
    output = np.dot(cnot_operator, qubit_state)
    return output
```

Similarly all the other basic gates for a double qubit system have been implemented. The list of all the double qubit gates is as follows:

DoubleQubitSystem(), printState(), cNotGate(), swapGate(), sqrtSwapGate(), sqrtISwapGate(), sycamoreGate(), czGate(), getState()

2.3 Multiple Qubit Systems

The library provides us the freedom to simulate a circuit with any number of qubits untill it is under the processing power of the personal computers. The multiple qubit system is one of the basic systems that one requires which simulating any circuit with real life implementations. To create a multiple qubit system, we first need to import the multiple_qubit_sytem.py file from the library into our project and call the constructor for the object. The constructor function takes in the number of qubits in our system as its parameter. If the required number of qubits is greater than 10, the simulator throws an exception saying that the maximum number of allowed qubits is 10. The implementation of the constructor function is as follows:

```
def __init__(self, n):
    if(n>10):
        raise Exception("The maximum permissble number of qubits is 10")
    else:
        self.num_of_qubits = n
        qubit_state = [[1],[0]]
        system_state = []
        for i in range(n):
            system_state.append(qubit_state)
        self.qubit_state = system_state
```

The inital state of all the qubits in the system is set as ket 0. This can be changed using one of the methods of the object.

2.3.1 Setting The States Of Qubits

We can easily set the states of the individual qubits of the systems through the setQubits() method of the object. We need to provide the qubit number of the specific qubit that we need to change the state of and the final probability amplitude of the ket 1 state that we want. The qubit number is to be provided in 1 based indexing. The implementation of the above is as follows:

```
def setQubits(self, qubit_num, one_state_value):
    if(self.num_of_qubits<qubit_num):
        raise Exception("Required Qubit number does not exist")
    else:
        zero_state_value = (1 - one_state_value**2)**0.5
        self.qubit_state[qubit_num-1] = [[zero_state_value], [one_state_value]]
```

This can be used to set any qubit state in the system. We can initialize the initial states of the qubit as we want in the start of computation or can change it mid computation as well.

2.3.2 Basic Gate Operations

All the basic single and double qubit gates have been implemented in the multi qubit system. We need to specify the qubit numbers that the gates have to be operated on in the parameters of the gate operation as well. The basic X gate operation is implemented as follows in the code:

```
def xGate(self, qubit_num):
    if(self.num_of_qubits<qubit_num):
        raise Exception("Required Qubit number does not exist")
    else:
        qubit = self.qubit_state[qubit_num-1]
        qubit_obj = singleQubit.SingleQubit(qubit[1][0])
        qubit_obj.xGate()
        self.qubit_state[qubit_num-1] = qubit_obj.getState()
```

The basic CNOT Ggate operation is implemented as follows in the code:

```
def cnotGate(self, qubit1, qubit2):
    if(self.num_of_qubits<qubit1 or self.num_of_qubits<qubit2):
```

```
            raise Exception("Required Qubit numbers does not exist")
        else:
            q1 = singleQubit.SingleQubit(self.qubit_state[qubit1-1][1][0])
            q2 = singleQubit.SingleQubit(self.qubit_state[qubit2-1][1][0])
            twoQubitSystem = doubleQubit.DoubleQubitSystem(q1, q2)
            twoQubitSystem.cnotGate()
            q1, q2 = twoQubitSystem.getState()
            self.qubit_state[qubit1-1] = q1
            self.qubit_state[qubit2-1] = q2
```

A brief list of all the methods available in the multiple qubits system is given below:

- MultipleQubitSystem(): This is the constructor method to initialize a qubit system. We have to pass the number of qubits that we need to have (restricted to 10), and the system is initialized with all the qubit states set as ket 0.
- setQubits(): This method is used to set the state of any specific qubit. We need to pass the qubit number and the probability amplitude of ket 1 that we need to set the qubit to.
- xGate(): This is the Pauli X Gate operation to be performed on a single qubit. We need to pass the qubit number on which the X gate has to be performed.
- yGate(): This is the Pauli Y Gate operation to be performed on a single qubit. We need to pass the qubit number on which the X gate has to be performed.
- zGate(): This is the Pauli Z Gate operation to be performed on a single qubit. We need to pass the qubit number on which the X gate has to be performed.
- hGate(): This is the Hadamard Gate operation to be performed on a single qubit. We need to pass the qubit number on which the X gate has to be performed.
- pGate(): This is the Phased Gate operation to be performed on a single qubit. We need to pass the qubit number on which the gate has to be performed on along with the phase parameter of the gate.
- virtualzGate(): This is the virtual Z Gate operation to be performed on a single qubit. We need to pass the qubit number on which the gate has to be performed.
- iGate(): This is the Identity Gate operation to be performed on a single qubit. We need to pass the qubit number on which the gate has to be performed.
- sGate(): This is the S Gate operation to be performed on a single qubit. We need to pass the qubit number on which the gate has to be performed.

- sDaggerGate(): This is the $S^\dagger$ Gate operation to be performed on a single qubit. We need to pass the qubit number on which the gate has to be performed.
- tGate(): This is the T Gate operation to be performed on a single qubit. We need to pass the qubit number on which the gate has to be performed.
- tDaggerGate(): This is the $T^\dagger$ Gate operation to be performed on a single qubit. We need to pass the qubit number on which the gate has to be performed on.
- uGate(): This is the Universal Gate operation with parameters θ, ϕ, and λ to be performed on a single qubit. We need to pass the qubit number on which the gate has to be performed on along with the parameters θ, ϕ, and λ.
- cnotGate(): This is the double qubit Controlled NOT Gate. We need to pass the qubit numbers of both the qubits on which the gate has to be performed on.
- swapGate(): This is the Swap Gate for a double qubit system. We need to pass the qubit numbers of both the qubits on which the gate has to be performed on.
- sqrtSwapGate(): This is the $\sqrt{SWAP}$ Gate for a double qubit system. We need to pass the qubit numbers of both the qubits on which the gate has to be performed on.
- sqrtISwapGate(): This is the $\sqrt{iSWAP}$ Gate for a double qubit system. We need to pass the qubit numbers of both the qubits on which the gate has to be performed on.
- sycamoreGate(): This is the Sycamore Gate operation for two qubits in the multi qubit system. We need to pass the qubit number of both the qubits on which the gate has to be performed on.
- czGate(): This is the double qubit Controlled Z Gate. We need to pass the qubit numbers of both the qubits on which the gate has to be performed on.
- printState(): This is the final method for a multiple qubit system. This helps to see the state of all the qubits in the system and prints them systematically to the console.

2.4 Non Ideal Qubits

In real life, there are a variety of factors affecting the state of the qubit such as the temperature, the driveline coupling issues, the noises. These try to bring down the total energy of the qubit down by transferring energy from the higher energy ket 1 state to the lower energy ket 0 state. These losses can be modelled as exponential decays with their own specific time constants and quality factors. There are 4 major types of losses affecting the state of the qubits. These are the driveline dissipation losses, driveline noise, intrinstic losses, and the

readout losses. For a standard qubit with frequency 5 Gigahertz, we have the values of the quality factors as:

- Driveline Dissipation: $Q_{Dd} = 5*10^7$
- Driveline Noise: $Q_{Dn} = 5*10^7$
- Intrinstic: $Q_I = 4*10^6$
- Readout: $\gamma_R = 250\text{-}500$ ns

The constructor function for the non-ideal qubit is implemented in the code as follows:

```
def __init__(self,  x, freq=5000000000):
    if(x<=1 and x>=0):
        y = (1 - x**2)**0.5
        self.qubit_state = [[y],[x]]
        self.omega = 2*np.pi*freq
        self.driveline_dissipation_qf = 50000000
        self.driveline_noise_qf = 50000000
        self.instrinsic_qf = 4000000
        self.readout_qf = 104719755119659774.61
    else:
        raise Exception("Enter a valid qubit state")
```

This function initializes the qubit and sets all the quality factors of the qubit according to standard values.

2.4.1 Effect Of Decoherence On The Qubit

The dissipation function for when the qubit is not under the effect of any gate operations has been modelled through the decoherenceDecay function that takes in the time for which the the decoherence has to be observed. The implementation of the function in the code is as follows:

```
def decoherenceDecay(self, time):
    a = self.qubit_state[0][0]
    b = self.qubit_state[1][0]
    qf_total = self.driveline_dissipation_qf + self.driveline_noise_qf
    + self.instrinsic_qf + self.readout_qf
```

```
        omega = self.omega
        time = time/1000000000
        a_final = (1 + (a**2-1)*np.exp(-qf_total*time/omega))**0.5
        b_final = ((b**2)*np.exp(-qf_total*time/omega))**0.5
        self.qubit_state = [[a_final],[b_final]]
```

2.4.2 X-Gate Operation With Decoherence

Under the influence of an ideal Pauli X Gate, the probability amplitudes of the qubit states vary according to absolute values of sinusoids. To account for decoherence while the gate operation is occuring on the qubit, we have to integrate the various factors affecting the qubit with time.

Our approach to modelling this was to divide the gate operation into smaller steps and then in each step, we calculate the change in the state of the qubit from the gate operation and then let it decay for the time step. This operation converges when the steps are large as it is basically the same as integration. The implementation of this in code is as follows:

```
    def xGate(self):
        steps = 1000
        gate_time = 50
        phase_steps = np.pi/(2*steps)
        time_steps = gate_time/steps
        b_start_value = self.qubit_state[1][0]
        a_start_value = self.qubit_state[0][0]
        f1 = np.cos(phase_steps)
        f2 = np.sin(phase_steps)
        for step in range(0, steps):
            b = self.qubit_state[0][0]
            a = self.qubit_state[1][0]
            if(a_start_value<0):
                a = -1*a
            if(b_start_value<0):
                b = -1*b
            b = b*f1 - a*f2
            a = a*f1 + b*f2
            b_start_value = b
            a_start_value = a
            self.qubit_state = [[abs(b)],[abs(a)]]
            self.decoherenceDecay(time_steps)
```

In a similiar way the Hadamard Gate has also been modelled. The X-Gate and the H-Gate are the founding pillars of the quantum circuits as all other gates can be written as a combination of these two gates.

2.4.3 U Gate Operation With Decoherence

The Universal Gate accept three parameters which represent rotation of the qubit on the bloch sphere representation. To model the non-ideal U Gate, we have taken a difference approach than the X Gate. We divided the U gate into a number of smaller U Gates which have been connected in series and at the end of each smaller gate operation we have taken the effect of decoherence. The implementation of this in code is as follows:

```
def uGate(self, theta, phi, lamda):
    steps = 1000
    gate_time = 50
    theta_steps = theta/steps
    phi_steps = phi/steps
    lamda_steps = lamda/steps
    time_steps  = gate_time/steps
    for step in range(0, steps):
        self.qubit_state = uGate.u_gate.u_gate(self.qubit_state, theta_steps,
        phi_steps, lamda_steps)
        self.decoherenceDecay(time_steps)
```

Chapter 3

Simulations and Results

3.1 Single Qubit System

We used the simulator to simulate a quantum circuit with just one qubit. We performed various gate operations on it and finally assessed the results. The code for the circuit is as follows:

```
import single_qubit as SQ
import numpy as np

single_qubit_object = SQ.SingleQubit(0.7071)
single_qubit_object.xGate()
single_qubit_object.tDaggergate()
single_qubit_object.pGate(np.pi/3)
single_qubit_object.printState()
```

The result of this simulation in the console was as follows:

```
(env) BTP\QuantumComputationLib>python testing.py
((0.7071+0j))|0} + ((0.6830192519526233+0.18301445697561486j))|1}
```

3.2 Double Qubit System

We used the simulator to simulate a quantum circuit with two qubits which were separately acted upon by single qubit gates and then created into a two qubit system, after which they were operated on double qubit gates. The code for the circuit is as follows:

```
import numpy as np
import QuantumComputationLib.single_qubit as qubit
import QuantumComputationLib.double_qubit_system as qubitSystem

q1 = qubit.SingleQubit(0.95)
q1.xGate()
q1.pGate(np.pi/3)
q1.hGate()

q2 = qubit.SingleQubit(0.8)
q2.yGate()
q2.sDaggerGate()
q2.hGate()

qubits = qubitSystem.DoubleQubitSystem(q1,q2)
qubits.cnotGate()
qubits.swapGate()
qubits.czGate()
qubits.printState()
```

The result of this computation comes out to be as follows in the console:

```
(env) BTP\QuantumComputationLib>python testing.py
((0.4400040232519076-0.3613250762860442j))|00}
+ ((-0.34632905327593155-0.23642511631807617j))|01}
+ ((-0.2236709467240682-0.5235748836819236j))|10}
+ ((-0.1299959767480922+0.39867492371395563j))|11}
```

3.3 Multiple Qubit System

The simulator was used to simulate a circuit with 6 qubits which were acted upon by both single and double qubit gates to test the simulation. The code implementation for the same is as follows:

```
import multiple_qubit_system as MQS

qubit_system = MQS.MultipleQubitSystem(6)
qubit_system.xGate(1)
```

```
qubit_system.xGate(6)
qubit_system.hGate(3)
qubit_system.setQubits(2, 1)
qubit_system.cnotGate(2,4)
qubit_system.printState()
```

The result of the simulation in the console comes out as:

```
(env) BTP\QuantumComputationLib>python testing.py
Printing State of all 6 qubits
Qubit 1----->
0.0|0} + 1.0|1}
Qubit 2----->
0.0|0} + 1.0|1}
Qubit 3----->
0.7071067811865475|0} + 0.7071067811865475|1}
Qubit 4----->
0.0|0} + 1.0|1}
Qubit 5----->
1|0} + 0|1}
Qubit 6----->
0.0|0} + 1.0|1}
```

3.4 Non Ideal Qubit

3.4.1 Decoherence Decay

According to the theoretical results, the qubit is supposed to decay, transferring energy from state ket 1 to ket 0. This brings an important discussion to the table limiting the time that we can keep the qubit idle without any computation. This is a really important factor to model and test as most of our quantum circuitry would depend on it. To test the effect of decoherence on the circuit, we ran the decoherence decay function for various amounts of time and checked the results. Each time the qubit state would start from ket 1 and would be left to decay for a short amount of time (in order or nanoseconds). The code implementation for the same is as follows:

```
import non_ideal_qubit as niq
```

```
time_durations = [1, 5, 10, 20, 50, 100, 500, 1000]
for time in time_durations:
    qubit_object = niq.NonIdealQubit(1)
    qubit_object.decoherenceDecay(time)
    qubit_object.printState()
```

The results from the simulation are as follows:

Time Duration (in ns)	Final Qubit State
1	$(0.05768694781990191)\|0> + (0.9983347214492863)\|1>$
5	$(0.12856339368055372)\|0> + (0.9917012926306685)\|1>$
10	$(0.18106324737509003)\|0> + (0.983471453805339)\|1>$
20	$(0.25395474996605355)\|0> + (0.967216100449987)\|1>$
50	$(0.39181408250533384)\|0> + (0.9200444145531799)\|1>$
100	$(0.5324177773735705)\|0> + (0.8464817247505034)\|1>$
500	$(0.9006244486327624)\|0> + (0.4345982081474022)\|1>$
1000	$(0.9820010217768836)\|0> + (0.18887560252493274)\|1>$

3.4.2 U Gate Operation

The Universal Gate operation on the qubit was built by splitting the parameters in the original U gate into smaller U gates and then having decoherence decay after each of the smaller operations. The simulation code for the U Gate is as follows:

```
import non_ideal_qubit as niq
import numpy as np

obj1 = niq.NonIdealQubit(1)
obj1.uGate(np.pi/2, np.pi/4, np.pi/8)
obj1.printState()
```

The results from the simulation are as follows:

```
(env) BTP\QuantumComputationLib>python testing.py
((0.09338007308216661-0.13266244921243248j))|0}
+ ((0.42326508251287187+0.8507753939294374j))|1}
```

3.4.3 X-Gate Operation

The effect of decoherence on X Gate can be seen through the following simulation:

```
import non_ideal_qubit as niq

obj2 = niq.NonIdealQubit(1)
obj2.xGate()
obj2.printState()
```

The output of the simulation comes out as:

```
(env) BTP\QuantumComputationLib>python testing.py
(0.9864949851652214)|0} + (0.158174105890255)|1}
```

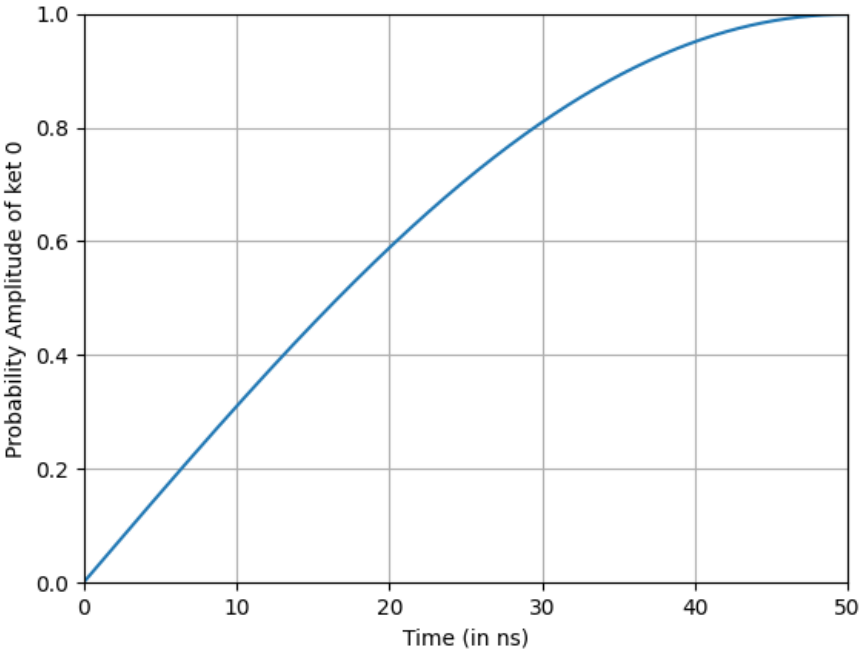


FIGURE 3.1: The time evolution of ket 0 state without decays

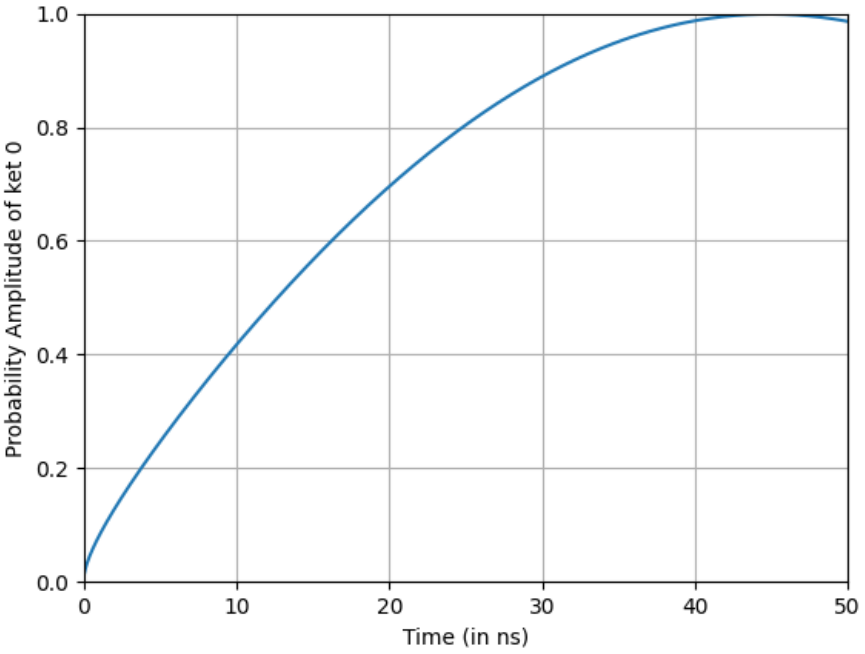


FIGURE 3.2: The time evolution of ket 0 state with decays

Due to the effect of decoherence, ket 0 state reaches its peak a lot earlier and then starts to go down again during the gate duration. This is an important observation in the field of quantum computing as this can be used to shorten the gate operation time and we can get the ket 0 state at its peak.

3.4.4 H-Gate Operation

The X Gate and the H Gate are one of the most basic gate operations in quantum computing. The reason being that these gates can be used to create all other quantum gates through combinations. The effect of decoherence on H Gate can be seen through the following simulation:

```
import non_ideal_qubit as niq

obj2 = niq.NonIdealQubit(1)
obj2.hGate()
obj2.printState()
```

The output of the simulation comes out as:

```
(env) BTP\QuantumComputationLib>python testing.py
```

```
(0.8528054427069179)|0} + (0.5215066646749492)|1}
```

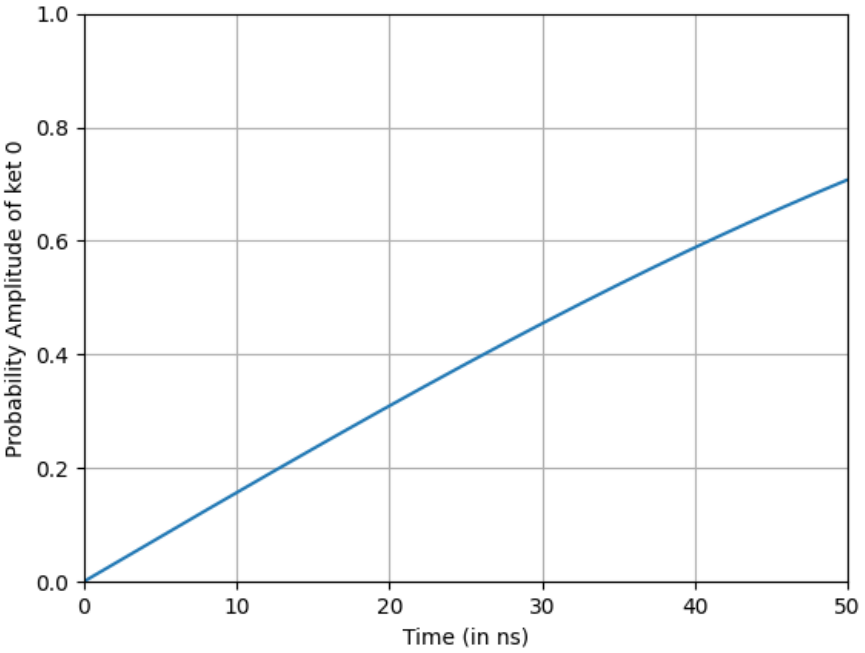


FIGURE 3.3: The time evolution of ket 0 state without decays

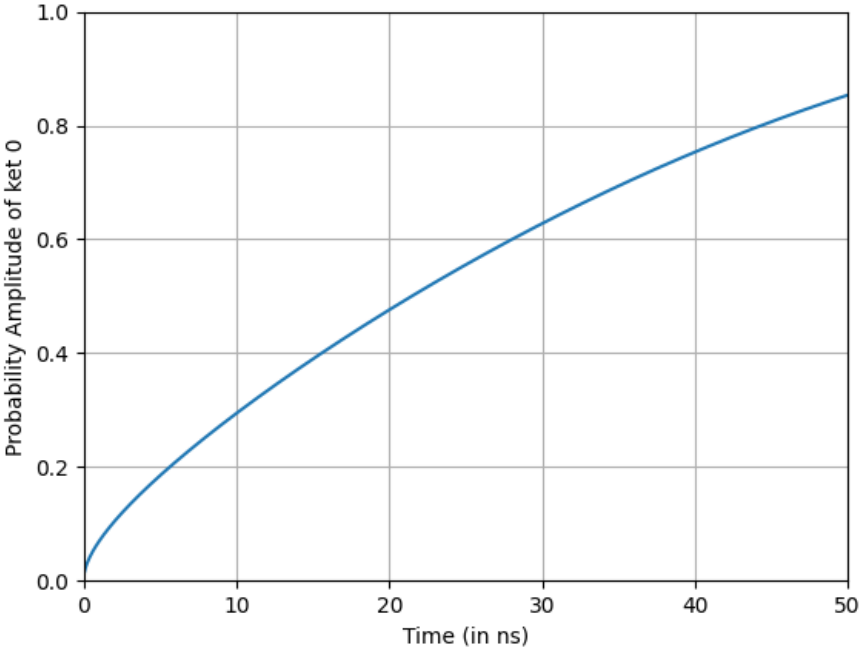


FIGURE 3.4: The time evolution of ket 0 state with decays

Due to the effect of decoherence, the probability amplitude of ket 0 state overshoots from its value, reaching a newer value during the gate operation.

Bibliography